Module 1 — Node.js Fundamentals & the Event Loop

Time: 1.5 hours · Covers **Practical 4** · **CO1**

> *Practical 4: Explain the concept of the event loop and its role in managing asynchronous tasks in Node.js.*

Part A — What is Node.js? (concept, ~15 min)

Node.js is a **runtime**: a program that executes JavaScript on your machine instead of in a browser. It uses Google's **V8** engine (the same one in Chrome) to run the code, and adds abilities the browser does not have — reading files, listening on network ports, talking to databases.

Two ideas make Node powerful:

1. **Single-threaded.** Your JavaScript runs on **one** thread. There is only one thing executing at any instant.
2. **Non-blocking / asynchronous.** Slow operations (reading a file, a network request, a DB query) do **not** freeze that thread. Node starts the operation, keeps running other code, and comes back to the result later.

How can one thread stay busy while waiting for slow work? That is the job of the **event loop**.

Part B — Synchronous vs Asynchronous (~15 min)

Synchronous code runs top to bottom, each line finishing before the next starts. If a line is slow, everything waits.

```
console.log("A");
console.log("B");
console.log("C");
// Output: A B C   (always, in order)
```

Asynchronous code hands slow work off and continues. The result arrives later, via a **callback**, **Promise**, or **async/await**.

```
console.log("A");
setTimeout(() => console.log("B"), 0); // "later", even with 0 ms delay
console.log("C");
// Output: A C B   <-- B is last, this surprises everyone at first
```

Why is `B` last? Because `setTimeout` schedules its callback to run **after** the current code finishes. That scheduling is the event loop at work.

Run it yourself: `code/sync-vs-async.js`

Part C — The Event Loop (~30 min)

Think of the event loop as a **manager with a to-do list** that never stops checking: *"Is the main code done? Is any finished background work ready to run?"*

Node keeps a few queues. The loop empties them in a fixed order. The order you must remember:

1. **Call stack** — the code running *right now*. Runs to completion first.
2. **Microtasks** — Promise callbacks (`.then` , `await`) and `process.nextTick` . Drained **completely** after the current code and **before** any timer.
3. **Macrotasks** — timers (`setTimeout` , `setInterval`) and I/O callbacks. Run **after** all microtasks are empty.

The classic ordering example

```
console.log("1: start");                        // sync -> call stack

setTimeout(() => console.log("2: timeout"), 0); // macrotask

Promise.resolve().then(() => console.log("3: promise")); // microtask

console.log("4: end");                          // sync -> call stack
```

Output:

```
1: start
4: end
3: promise
2: timeout
```

Reasoning:

- `1` and `4` are plain synchronous code → run immediately, in order.
- `3` is a Promise microtask → runs right after sync code, before any timer.
- `2` is a timer macrotask → runs last, even with `0` ms.

Run and study: `code/event-loop.js`

Why this matters

Because Node is single-threaded, **long synchronous code blocks everything** — including the event loop, so no timers or requests get handled. The whole point of async APIs is to keep the loop free. Compare:

- Blocking: `code/blocking.js` — freezes for 3 seconds.
- Non-blocking: `code/non-blocking.js` — stays responsive.

Part D — Three ways to handle async results (~20 min)

Same task (wait, then produce a value), written three ways.

1. Callbacks (oldest style)

```
function getUser(id, callback) {
  setTimeout(() => callback(null, { id, name: "Ada" }), 500);
}
getUser(1, (err, user) => {
  if (err) return console.error(err);
```

```
  console.log(user);
});
```

Nesting many callbacks becomes "callback hell". Promises fix that.

2. Promises

```
function getUser(id) {
  return new Promise((resolve) => {
    setTimeout(() => resolve({ id, name: "Ada" }), 500);
  });
}
getUser(1).then((user) => console.log(user));
```

3. async / await (modern, cleanest — use this)

```
async function main() {
  const user = await getUser(1); // "pause here until the Promise resolves"
  console.log(user);
}
main();
```

`await` does **not** block the thread — the event loop keeps running other work while this function is paused.

Full comparison: `code/async-styles.js`

Summary

- Node runs JavaScript on **one thread** using V8.
- Slow work is **non-blocking**; results come back later.
- The **event loop** orders that work: sync code → microtasks (Promises) → macrotasks (timers/IO).
- Never block the loop with long synchronous work.
- Prefer **async/await** for readable asynchronous code.

Now do `practice.md` .